Project KIT — Complete System Handbook

This is the primary, plain-language guide to Project KIT. It is written for a teacher, school client, new developer, maintainer, or anyone who wants to safely change the system. The descriptions below are based on the current source code, not on an idealized future design.

1. What Is Project KIT?

Project KIT is a school assessment and academic-analysis platform. It helps a school organize its academic setup, create assessments, collect student answers, verify scanned answer sheets, calculate results, inspect question quality, understand competency performance, and plan follow-up support.

It is intended for teachers and academic staff, school administrators, and students who take part in a remediation/reassessment workflow. A tenant is the school or organization boundary used to keep one school's users and records separate from another's.

Project KIT solves these practical problems:

- keeping sections, subjects, strands, faculty, and students in one configuration;
- defining an assessment and its answer key;
- accepting answer data and OMR scans;
- keeping uncertain scans in a teacher verification queue;
- producing repeatable scores and class summaries;
- showing item difficulty, discrimination, distractor use, and competency mastery;
- identifying deterministic learning weaknesses;
- supporting teacher-reviewed remediation and reassessment; and
- giving teachers an AI assistant that can explain and retrieve permitted information.

The frontend is the part a person sees: HTML pages, forms, navigation, tables, charts, and the AI chat panel. The backend is the protected server: it authenticates users, enforces tenant scope, validates input, reads/writes the database, performs official calculations, and coordinates AI tools. Prisma is the database access library; the schema is in `backend/prisma/schema.prisma`.

2. Project KIT at a glance

```
Teacher
   ↓
Project KIT
   ↓
Academic setup
   ↓
Students / sections / subjects
   ↓
Assessment
   ↓
Questions + TOS + answer key
   ↓
Student answers / OMR
   ↓
Verification (when needed)
   ↓
```

```
Evaluation
   ↓
Deterministic mathematical calculations
   ↓
Results
   ↓
Item analysis
   ↓
Competency analysis
   ↓
Weakness detection
   ↓
Teacher-reviewed remediation
   ↓
Reassessment
```

Each arrow is a hand-off, not an AI guess. The backend persists response rows, calculates official values, and returns them through APIs. The frontend displays those values.

Technology in everyday terms

The browser side uses plain JavaScript modules, Alpine.js for lightweight interaction, Webpack to build the pages, ApexCharts for charts, Tesseract.js assets for the exam-import UI, and Jest for frontend tests. The server uses NestJS (the HTTP/module framework), TypeScript, Prisma (database access), PostgreSQL (the schema's database provider), Jest, session/role guards, CSRF protection, Helmet security headers, Redis health/rate-limit support, and an optional Ollama/OpenAI-compatible AI provider. These tools support the product; a teacher does not need to know them to use the website.

3. How a teacher actually uses Project KIT

Step 1 — Set up academic information

What is it? The school's working academic context: faculty information, school year, term, strands, sections, subjects, and student roster.

Why use it? Assessments and tenant security need to know which school, section, subject, and students are involved.

How does it work? The configuration screen calls `GET /api/config` and `PUT /api/config`. `backend/src/config/config.service.ts` normalizes the submitted shape and writes the tenant's `Config` row plus related `Faculty`, `Strand`, `Section`, `Subject`, and `Student` rows in Prisma. `frontend/src/js/stores/config-store.js` loads the server copy and caches it for the browser interface. A failed save may leave a clearly warned local-only copy; that copy is not shared database authority.

Where do I change it? Backend: `backend/src/config/config.controller.ts` and `config.service.ts`. Frontend: `frontend/src/js/views/config.js`, `frontend/src/js/api/config-api.js`, and `frontend/src/js/stores/config-store.js`. Database: the `Config` and related models in `backend/prisma/schema.prisma`.

Step 2 — Create an assessment

What is it? A named test tied to one subject and section for a term and school year.

Why use it? It gives every answer, attendance record, result, and analysis a single assessment identity.

How does it work? The teacher supplies a title, `subjectId`, `sectionId`, term, school year, optional target item count, and optional passing MPS. `POST /api/assessments` is handled by `AssessmentsController` and `AssessmentsService`. The service verifies that subject and section belong to the tenant, creates a DRAFT assessment, and creates attendance rows for the section's students.

Where do I change it? `backend/src/assessments/assessments.controller.ts`, `assessments.service.ts`, and DTOs in `backend/src/assessments/dto/`; frontend `frontend/src/js/views/assessments.js` and `frontend/src/js/api/assessment-api.js`.

Step 3 — Create the Table of Specifications (TOS)

What is it? A TOS is a worksheet saying what a test is intended to cover. Its rows contain competency/topic descriptions, hours, and item allocation information.

Why use it? It connects test items to what the teacher intended to measure, so results can be summarized by competency instead of only by question number.

How does it work? The active editor is `frontend/src/js/views/tos.js`. It loads competency data, validates worksheet totals, and saves rows through `/api/tos/documents`. `backend/src/tos/tos.service.ts` stores the rows as JSON in `TosDocument`, validates the subject/term/school-year context, and can link a document to an assessment. During competency results, `ResultsService` reads `itemNumbers` (or `items`) from those rows. If no usable mapping exists, it groups items into five-item blocks such as `Items 1–5`.

The current implementation does not calculate a separate Bloom's-taxonomy statistic or cognitive-level score. If a row contains extra fields, they are stored as row data, but the results calculator uses competency names and item-number mappings.

Where do I change it? Frontend `frontend/src/js/views/tos.js`; backend `backend/src/tos/tos.controller.ts` and `tos.service.ts`; persistence model `TosDocument` in the Prisma schema; mapping consumption `backend/src/results/results.service.ts`.

Step 4 — Create or import the answer key

What is it? The answer key is the expected answer for each item. The active answer-key service accepts A, B, C, or D and normalizes letters to uppercase.

Why use it? A response becomes correct only by comparison with the stored key.

How does it work? The key can be represented as an array of objects or an object map. Conceptually:

```
Question 1 → A
Question 2 → C
Question 3 → B
```

`AnswerKeysService.parseAnswers()` validates item numbers and letters, removes duplicate item numbers, and sorts them. `grade()` compares each student's answer with the key, counts correct answers, creates `Response` / `ResponseItem` data, and records a `GRADED` response. Assessment-linked answer keys also use the assessment endpoints.

Where do I change it? `backend/src/answer-keys/answer-keys.service.ts`, `answer-keys.controller.ts`, and `answer-keys.module.ts`; frontend `frontend/src/js/api/answer-key-

`api.js` and `frontend/src/js/components/assessment/setup-answer-key.js`.

Step 5 — Collect student answers

The repository supports several answer paths:

- answer-key grading through the answer-key API;
- response/config APIs used by the import and response stores;
- OMR form generation and scan submission; and
- scan verification before an evaluated response is created.

The OMR path is:

```
Paper form
  ↓
Image payload
  ↓
Deterministic OMR processor
  ↓
Detected choice, blank, ambiguity, or multiple mark
  ↓
Verification queue when review is needed
  ↓
Teacher review
  ↓
EVALUATED response
```

`backend/src/omr/omr.controller.ts` exposes form, scan, queue, and review routes. `omr-scanner.service.ts` persists scan jobs/results and changes a reviewed response to `EVALUATED`. `omr-processor.service.ts` validates a base64 image, calculates a SHA-256 image hash, reads the canonical template, reports confidence/state, and returns normalized answers. Its current bubble sampling is deterministic hash-based pseudo-sampling for the implemented/testing path; it is not a full production computer-vision/OCR engine. PDF/DOCX AI uploads likewise use basic readable-text extraction, not a full document parser.

Where do I change it? `backend/src/omr/omr-processor.service.ts`, `omr-scanner.service.ts`, `omr-form.service.ts`, and `omr-template.config.ts`; frontend `frontend/src/js/views/assessment-workspace.js` and `frontend/src/js/api/assessment-api.js`.

Step 6 — Verification

What is it? Teacher confirmation of an uncertain scan.

Why use it? A blank, multiple mark, low-confidence detection, or incorrect student identity should not silently become an official result.

How does it work? Scan results may be stored as `VERIFICATION_REQUIRED`, with queue items in `PENDING`. The teacher reviews or overrides the answer. The scanner service stores the final answer data and creates an `EVALUATED` response when the review is accepted. The queue is available at `/api/assessments/:id/verification-queue` and its review subroute.

Where do I change it? `backend/src/omr/omr-scanner.service.ts` and `omr.controller.ts`; the workspace review UI is in `frontend/src/js/views/assessment-workspace.js`.

Step 7 — Evaluation and official results

There are three useful distinctions:

- **Raw response:** answer data before a trusted final evaluation.
- **Graded response:** an answer-key grading operation has produced a score; the answer-key service writes status `GRADED`.
- **Evaluated response:** the official results path accepts the row for analysis; `ResultsService.getVerifiedResponses()` filters `status: 'EVALUATED'`.

The results endpoints therefore do not treat every saved answer as official. They load tenant-scoped evaluated `Response` rows and their `ResponseItem` rows, then call `backend/src/results/results-calculator.ts`.

Important API map

An API is the structured conversation between the browser and server. The main active routes are:

Route	What it is for
`POST/GET /api/auth/*`	register, login, session, logout, password flows, and CSRF token
`GET/PUT /api/config`	load/save tenant academic configuration
`GET/POST/PATCH/PUT /api/assessments/*`	create, list, open, status, attendance, answer-key, score-override, and response workspace operations
`GET /api/assessments/:id/results/summary`	class summary and item-quality summary
`GET /api/assessments/:id/results/students`	ranked student results
`GET /api/assessments/:id/results/items`	item analysis
`GET /api/assessments/:id/results/competencies`	competency analysis
`GET /api/assessments/:id/results/export/csv`	CSV export
`/api/tos/*`	competency/TOS documents
`/api/config/responses/*`, `/api/imports`	response/config-row compatibility and import records used by the active stores
`/api/answer-keys/*`	list, save, grade, and delete answer keys
`/api/assessments/:id/omr-form/*` and `/:id/scan*`	OMR forms, scans, queue, and review
`/api/assessments/:id/intelligence/*` and `/api/intelligence/*`	weakness, remediation, practice, reassessment, and progress workflow
`/api/ai/chat`, `/api/ai/stream`, `/api/ai/upload`, `/api/ai/info`	AI assistant and teaching-module context

`/api/admin/*`, `/api/users/*`, `/api/recommendation/*`	administration, user, and recommendation operations

The exact method, DTO, guard, and response wrapper are defined in the controller files; do not invent a route based only on a page name.

4. The math — a friendly explanation

All official score and psychometric values described here are deterministic backend calculations. The AI may explain a value, but it does not generate the value. The central implementation is `backend/src/results/results-calculator.ts`; answer-key scoring begins in `backend/src/answer-keys/answer-keys.service.ts`, and learning-status thresholds are in `backend/src/intelligence/intelligence-calculator.ts`.

5. Basic student score

What is it? The number of correct answers.

Why use it? It is the simplest measure of how many items the student answered correctly.

Formula. `score = number of correct answers`.

Symbols. Each item contributes `1` when correct and `0` otherwise; adding those values gives the score.

Example. Ten questions and seven correct answers produce a score of 7.

Meaning. The student got seven items right. It says nothing about national standing or why an answer was wrong.

Where is it implemented? `AnswerKeysService.grade()` and the `ResultsCalculator` response records.

6. Passing percentage / MPS

What is it? MPS (`passingMps`) is the minimum percentage required to pass an assessment.

Why use it? It turns a percentage into a pass/fail status for that assessment.

Formula. `percentage = score / totalItems × 100`; pass when `percentage ≥ passingMps`.

Example. A student at 72% with MPS 75% is `FAILED`; a student at 75% passes.

Important implementation detail. New assessments default `passingMps` to `60.0` in `AssessmentsService.create()` when no value is supplied. `ResultsService` uses `assessment?.passingMps || 75` when reading results, so a missing/falsy stored value falls back to 75. These are different defaults and must not be silently normalized by documentation or a future change.

Where is it implemented? Assessment creation in `backend/src/assessments/assessments.service.ts`; pass/fail and percentage in `ResultsCalculator.calculateStudentPerformance()` and class passing-rate logic.

7. Mean

What is it? The average score of a group.

Why use it? It gives a single typical score for the evaluated class.

Formula. `mean = (all scores added together) / (number of students)`.

Symbols. `Σ` means “add all”; `N` is the number of scores.

Example. Scores 60, 70, and 80: `(60 + 70 + 80) / 3 = 70`.

Meaning. The class averaged 70 points. It does not mean every student scored 70.

Where is it implemented? `ResultsCalculator.calculateClassStatistics()`.

8. Median

What is it? The middle score after sorting scores from low to high.

Why use it? It describes the middle student and is less affected by one unusually high or low score than the mean.

Formula. Odd count: take the middle value. Even count: average the two middle values.

Example. `40, 60, 70, 80, 100` has median 70. For `40, 60, 80, 100`, the median is `(60 + 80) / 2 = 70`.

Meaning. Half the evaluated scores are at or below the median and half are at or above it, allowing for ties.

Where is it implemented? `ResultsCalculator.calculateClassStatistics()`.

9. Standard deviation

What is it? A measure of how spread out scores are around the mean.

Why use it? Two classes can have the same average but very different consistency.

Formula used by Project KIT. `population variance = Σ(score - mean)² / N`; `population standard deviation = √variance`.

Symbols. The squared difference measures each score's distance from the mean; `N` is the whole evaluated group. Dividing by `N` means this is population, not sample, standard deviation.

Example. Class A `69, 70, 71` has a small spread. Class B `30, 70, 100` has a large spread even though both examples center near 70.

Meaning. A small number means scores cluster together; a larger number means students are more spread out. It does not label a student as good or weak by itself.

Where is it implemented? `ResultsCalculator.calculateClassStatistics()`.

10. Rank

What is it? A student's relative order by score within this evaluated assessment.

Why use it? It lets a teacher compare students in the same assessment group.

How it works. Scores are sorted descending. Rank is `1 + the number of scores strictly greater than this score`.

Example. Alice 90 → rank 1; Bob 90 → rank 1; Chris 80 → rank 2; David 70 → rank 3. Tied 90s share rank 1, and the next rank is 2: this is dense ranking.

Meaning. Rank is local to the evaluated assessment group, not a national ranking.

Where is it implemented? `ResultsCalculator.calculateStudentPerformance()`.

11. Percentile

What is it? Project KIT's percentile estimates a student's relative position within the evaluated assessment group.

Why use it? It communicates position while giving tied students a shared mid-position.

Formula. `percentile = (L + 0.5S) / N × 100`, where `L` is the number with lower scores, `S` is the number tied at the same score, and `N` is the number evaluated.

Example. For scores `8, 8, 5`, an 8 has `L=1`, `S=2`, `N=3`, so percentile is `(1 + 1) / 3 × 100 = 66.7`. The 5 has `L=0`, `S=1`, so 16.7.

Meaning. This is not a national percentile, standardized ranking, IQ-like score, or norm-referenced result. It is a relative position inside this result set.

Where is it implemented? `ResultsCalculator.calculateStudentPerformance()`.

12. Item difficulty index

What is it? The proportion of evaluated students who answered one question correctly.

Why use it? It shows whether an item was easy or difficult for this particular group.

Formula. `p = students correct / students evaluated`.

Example. `18 / 30 = 0.60`, displayed as 60%.

Important naming detail. In classical item analysis, a higher difficulty index `p` means the question was easier because more students got it right. Project KIT's bands are: `p ≤ .20` VERY_DIFFICULT; `.20 < p ≤ .40` DIFFICULT; `.40 < p ≤ .60` MODERATE; `.60 < p ≤ .80` EASY; `p > .80` VERY_EASY.

Meaning. A low value invites a content/difficulty review; a high value may mean the item is accessible or too easy. It is not a judgment without context.

Where is it implemented? `ResultsCalculator.calculateItemAnalysis()`.

13. Discrimination index

What is it? A check of whether a question separates students who performed well overall from students who performed poorly overall.

Why use it? A useful item is generally answered correctly more often by the stronger-performing group.

Formula. Sort students by total score. Take the upper and lower groups, each `floor(27% of N)` with a minimum group size of 1. `D = proportion correct in upper group - proportion correct in lower group`.

Example. Upper group: `7/8 = 87.5%`; lower group: `3/8 = 37.5%`; `D = 87.5% - 37.5% = 50%` (stored as about `.500`).

Meaning. Positive is generally encouraging; near zero is questionable; negative means lower scorers did better and is a warning to inspect the item, key, or scoring.

Edge cases. Project KIT does not calculate this value for fewer than 10 evaluated students; it returns `null` with `INSUFFICIENT_SAMPLE`. Ties at the group boundary follow the sorted array order.

Where is it implemented? `ResultsCalculator.calculateItemAnalysis()`.

14. Point-biserial correlation

What is it? A statistic checking whether students who get one item right also tend to perform better on the rest of the test.

Why use it? It is a second item-quality signal. It can reveal an item whose correct option does not align with overall performance.

Formula used. `r_pbi = ((M_p - M_q) / s_t) × √(p × q)`, where `p` is item difficulty, `q = 1-p`, `M_p` is the mean corrected score for students who got the item right, `M_q` is the mean corrected score for students who did not, and `s_t` is the corrected-score population standard deviation.

Why remove the current item? The implementation subtracts the current item's one point from each student's total before computing the comparison. That prevents the item from artificially helping both its own correctness and the total used to judge it.

Conceptual example. If correct-answer students average 14 on the remaining items and incorrect-answer students average 10, the positive difference suggests the item aligns with broader performance. A negative difference suggests inspection is needed.

Meaning. Positive values are usually useful; negative values are a warning. It is not a grade or a causal claim.

Edge cases. Fewer than 10 students gives `INSUFFICIENT_SAMPLE`. All-correct/all-wrong items (`p=0` or `p=1`) or zero corrected-score spread produce `UNDEFINED` and no numeric value.

Where is it implemented? `ResultsCalculator.calculateItemAnalysis()`.

15. Distractor analysis

What is it? Analysis of the wrong choices offered by a multiple-choice item. A distractor is a wrong answer option.

Why use it? Wrong options show common misunderstandings. An option chosen by almost nobody may not be plausible enough to test understanding.

Formula. For each A, B, C, D, and BLANK: `option percentage = option count / evaluated students × 100`.

Example. If 1 of 50 students chooses wrong option C, C is selected by 2%.

Project KIT rule. A non-correct A–D distractor is `weak` when its percentage is below 5%. BLANK is counted but is never marked as a weak distractor. Unknown answer letters are counted in the blank bucket.

Meaning. A weak distractor is a prompt to inspect wording and alternatives; it is not proof that the question is invalid.

Where is it implemented? `ResultsCalculator.calculateItemAnalysis()`.

16. Item quality

What is it? A rule-based label combining item signals.

Why use it? It helps a teacher decide which questions deserve review.

How does it work? Rules are evaluated in this order:

1. Fewer than 10 students → `INSUFFICIENT_SAMPLE`.
2. Negative discrimination or negative point-biserial → `POOR`.
3. Discrimination below `.20`, point-biserial below `.15`, or any weak distractor → `REVIEW`.
4. Difficulty between `.30` and `.80`, discrimination at least `.40`, and point-biserial at least `.30` → `EXCELLENT`.
5. Otherwise → `GOOD`.

What should a teacher do? For `POOR`, inspect the key, wording, ambiguity, and whether the item measures the intended competency. For `REVIEW`, inspect distractors and the two item statistics. `GOOD` needs no automatic action. `EXCELLENT` meets all current rule conditions but is still a statistical signal, not a guarantee of content quality. `INSUFFICIENT_SAMPLE` means wait for more data rather than judging the item.

Where is it implemented? `ResultsCalculator.calculateItemAnalysis()`.

17. Competency mastery

What is it? A group summary of correct responses for questions mapped to one competency.

Why use it? A teacher can see which learning area needs attention instead of reading only individual item rows.

Formula. `mastery percentage = correct mapped item responses / total mapped item responses × 100`.

Example. Photosynthesis: five mapped questions, three correct response events → `3/5 = 60%` for one student; class mastery uses all response events for the mapping.

Project KIT labels. `<60` → `NO_MASTERY`; `60 to <75` → `NEAR_MASTERY`; `≥75` → `MASTERY`.

Meaning. These are application thresholds in the code, not automatically national standards or curriculum certification.

Where is it implemented? Mapping extraction and five-item fallback are in `backend/src/results/results.service.ts`; calculation is `ResultsCalculator.calculateCompetencyAnalysis()`.

18. Student intelligence and weakness analysis

What is it? Deterministic labels that turn an individual student's competency results into support priorities.

Why use it? It helps a teacher decide which student/competency combination needs follow-up.

How does it work? `IntelligenceCalculator` uses defaults: `≥80` MASTERED, `≥60` DEVELOPING, `≥40` NEEDS_SUPPORT, and below 40 CRITICAL. Priorities map to LOW, MEDIUM, HIGH, and CRITICAL respectively. It also creates evidence text, lists critical/developing/mastered competencies, and chooses the highest overall priority present.

Growth formula. `growth points = latest mastery - previous mastery`.

Example. Previous 52%, latest 60% → `+8.0` percentage points and `IMPROVED`. More than +5 is improved; less than -5 is declining; otherwise stable. Latest mastery at least 80 overrides the progress status to `MASTERED`. No previous value gives `INSUFFICIENT_DATA`, unless latest is already at least 80.

Meaning. These labels guide support. They do not change a student's official score or pass/fail result.

Where is it implemented? `backend/src/intelligence/intelligence-calculator.ts`; workflow persistence and AI remediation calls are in `backend/src/intelligence/intelligence.service.ts` and `ai-remediation.service.ts`.

19. What is not implemented

The audited code does not currently implement Cronbach's alpha, IRT, Rasch modeling, standard error of measurement, national norms, scaled scores, or AI-generated official scores. Do not document or build on those as if they already exist.

20. The AI — explained for a client

Project KIT's AI is an assistant, not the grading authority. The backend calculators produce official score, pass/fail, rank, percentile, item, competency, and quality values. The AI can retrieve permitted results and explain or suggest actions, but it must not replace those calculations.

21. What the AI can do

The active AI can answer questions, interpret permitted Project KIT data, orient itself to the current page, use an optional assessment context, retrieve information through permission-checked tools, explain assessment results, identify data-backed areas for attention, and support teacher-reviewed remediation proposals. It can also accept an uploaded teaching module for indexed context.

The public backend entry points are `POST /api/ai/chat`, `POST /api/ai/stream`, `POST /api/ai/upload`, and `GET /api/ai/info`, implemented in `backend/src/ai/ai.controller.ts`. The frontend callers are in `frontend/src/js/api/ai-api.js` and the active interface is `frontend/src/js/ai-modal.js`.

22. What the AI cannot do

The AI does not determine official student scores, correctness, pass/fail, rank, percentile, difficulty, discrimination, point-biserial, competency mastery, or official psychometric results. It cannot grant itself tenant access or bypass tools. AI-generated remediation/reassessment material is a proposal in a teacher-controlled workflow; it is not automatically an official assessment.

23. AI flow

```
User asks a question
      ↓
Frontend sends message, page, and optional assessment ID
      ↓
Backend session/tenant guard checks access
      ↓
AiService resolves page and assessment context
      ↓
AiModelService infers intent (or rule router falls back)
      ↓
Planner selects permissioned tools
      ↓
Tools/repositories retrieve allowed data
```

```
      ↓
Response synthesizer creates assistance text
      ↓
Frontend displays normal response or SSE stream
```

Non-streaming `/chat` returns `message`, nested `response.intent`, `parameters`, `plan`, `execution`, and `context.history/role`. Streaming emits events including `message_start`, `context_resolved`, `tool_start`, `tool_result`, `thinking_start`, `answer_delta`, `message_complete`, and `error`. If no AI provider is configured, rule-based intent routing is used. Provider failures fall back to rule routing or deterministic response synthesis; an unavailable provider does not change stored assessment data.

24. AI and privacy

The AI request can include the user message, page context, assessment ID, conversation/stream fields, tenant role context, permitted tool results, and optional indexed upload content. The controller receives the authenticated session before service work. Passwords, cookies, and session tokens are not fields in `AiRequestDto` and are not directly sent as chat content by the frontend.

When an external provider is configured, the assembled intent/synthesis prompt and permitted contextual data can leave the backend for that provider. The code supports an Ollama-native endpoint or an OpenAI-compatible endpoint using environment configuration. The implementation does not promise citations, national data, or a particular provider's retention policy; those require deployment/provider review.

25. How the database works

```
Tenant
 ├── Config (the default academic configuration)
 │    ├── Faculty
 │    ├── Strands
 │    ├── Sections
 │    ├── Subjects
 │    └── Students
 │
 └── Assessments
      ├── Attendance
      ├── AnswerKey / AnswerKeyVersion
      ├── Response → ResponseItem
      ├── OMR forms, scans, verification items
      ├── TOS document
      ├── audit records
      └── remediation / reassessment records
```

What is a tenant? A school/organization boundary. **Why use it?** Every normal service query should stay inside that boundary. **How?** `TenantContext`, `SessionGuard`, and tenant-scoped Prisma `where` clauses are used by controllers/services. **Where?** `backend/src/common/tenant-context.ts`, `backend/src/auth/session.guard.ts`, and each domain service.

The main Prisma entities are `Tenant`, `User`, `Role`, `Session`, `RefreshToken`, `Config`, `Faculty`, `Strand`, `Section`, `Subject`, `Student`, `Assessment`, `Attendance`, `AnswerKey`, `TosDocument`, `Response`, `ResponseItem`, OMR entities, `RemediationPlan`, `ReassessmentProposal`,

`CompetencyProgress`, `Recommendation`, `AuditLog`, and `AppSettings`. The schema and relationships are authoritative in `backend/prisma/schema.prisma`.

26. Frontend versus backend

Frontend

What is it? The browser application built from static HTML and ES modules by Webpack.

Why use it? It gives teachers pages, forms, navigation, charts, stores, and feedback.

How does it work? `frontend/src/js/index.js` is the bundle entry. It initializes authentication, stores, navigation, AI, and page views; PJAX swaps page content and reinitializes views. API clients in `frontend/src/js/api/` call the backend. Views in `frontend/src/js/views/` display data and collect input.

Where do I change it? `frontend/webpack.config.js`, `src/js/index.js`, the relevant view/API/store/component. The active AI UI is `src/js/ai-modal.js` plus active chat components.

Backend

What is it? The NestJS server and its domain modules.

Why use it? It is the security and data authority and keeps calculations consistent for every client.

How does it work? The normal path is Controller → Service → PrismaService/database. Results call calculator classes; AI calls permissioned tools and repositories.

Where do I change it? Feature folders under `backend/src/`; module registration is in `backend/src/app.module.ts`; startup/security middleware is in `backend/src/main.ts`.

The frontend must not grow a second official scoring engine. It may format or chart returned values, but calculation changes belong in backend services/calculators and their tests.

Authentication, roles, and tenant safety

What are they? Authentication proves who signed in; authorization decides what that role may do; tenant scope decides which school's records may be seen.

Why use them? A teacher should not read another school's students, and an administrator route should not become a teacher route by accident.

How do they work? `main.ts` configures cookies, CORS, Helmet, double-submit CSRF protection, and rate limiting. Auth controllers create/check sessions; `SessionGuard` attaches authenticated context; role decorators/guards and service checks enforce role-sensitive operations. Services repeatedly include tenant/config conditions in Prisma queries.

Where do I change them? Backend `backend/src/auth/`, `backend/src/common/tenant-context.ts`, `backend/src/common/request-with-tenant.ts`, `backend/src/main.ts`, and the relevant domain service. Frontend session/API behavior is in `frontend/src/js/auth-service.js`, `auth-adapter.js`, and `api/api-client.js`. Treat these files as security-sensitive.

27. Where to tinker

If you want to change...	Start here

Student score, percentage, pass/fail, rank, percentile	`backend/src/results/results-calculator.ts`, especially `calculateStudentPerformance()`; answer-key grading in `backend/src/answer-keys/answer-keys.service.ts`
MPS source/default	`backend/src/assessments/assessments.service.ts` (`passingMps` default) and `backend/src/results/results.service.ts` (read fallback)
Mean, median, standard deviation, passing rate	`ResultsCalculator.calculateClassStatistics()`
Difficulty, bands, discrimination, point-biserial, distractors, item quality	`ResultsCalculator.calculateItemAnalysis()`
Competency mastery	`backend/src/results/results.service.ts` mapping logic and `ResultsCalculator.calculateCompetencyAnalysis()`
Intelligence thresholds, priorities, growth	`backend/src/intelligence/intelligence-calculator.ts`
AI orchestration/provider/fallback	`backend/src/ai/ai.service.ts`, `ai-model.service.ts`, `ai.controller.ts`
AI tools/permissions/planning	`backend/src/ai/framework/`, `backend/src/ai/framework/tools/`, and AI repositories
Active AI UI/request/stream rendering	`frontend/src/js/ai-modal.js`, `frontend/src/js/api/ai-api.js`, active `frontend/src/js/chat/` files
Assessment screens/API	`frontend/src/js/views/assessments.js`, `assessment-workspace.js`, `frontend/src/js/api/assessment-api.js`; backend `src/assessments/`
TOS editor/storage	`frontend/src/js/views/tos.js`; backend `src/tos/`
OMR form/scan/verification	backend `src/omr/`; frontend assessment workspace/API
API request formatting	`frontend/src/js/api/` and `api-client.js`
Database relationships	`backend/prisma/schema.prisma`
Authentication, roles, CSRF/session behavior	`backend/src/auth/`, `backend/src/main.ts`, frontend `auth-service.js`/`auth-adapter.js`
Tenant isolation	`backend/src/common/tenant-context.ts`, `request-with-tenant.ts`, `SessionGuard`, and domain services
Charts and visual formatting	relevant active view and `frontend/src/js/chat/components/chart.js` only where imported; do not use archived renderer components

28. How to change a calculation safely

```
1. Find the calculation and its caller.
2. Read the existing calculator tests.
3. Read the matching section of this handbook and CALCULATIONS.md.
4. Write down the current formula, rounding, defaults, and edge cases.
5. Decide the desired behavior and its user meaning.
6. Change the backend implementation.
7. Update or add backend tests with concrete examples.
8. Update PROJECTKIT.md and CALCULATIONS.md.
9. Run backend tests, build, and Prisma validation if schema-related.
10. Run frontend tests/build if returned data or UI changed.
11. Review the API result and full Git diff.
```

Do not change a calculation only in a chart or browser view. The backend is where every client receives the same official result.

29. How to change the AI safely

1. Identify whether the change is AI UI, request shape, intent routing, planner, tool, permissions, provider adapter, prompt, or response formatting.
2. Start at the active `AiController` / `AiService` path and trace the request.
3. Check tenant and role permissions before adding a tool or data field.
4. Check exactly what data can reach the provider.
5. Preserve the rule-based/provider failure fallback.
6. Run the AI framework/service tests and backend build.
7. Confirm that the AI still cannot override deterministic results or perform hidden writes.

30. How to add a new feature

Use the existing boundary:

```
Frontend UI
    ↓
Frontend API client
    ↓
Backend controller + DTO validation
    ↓
Backend service
    ↓
PrismaService / repository
    ↓
Database
```

Put input validation in DTOs/services, tenant authorization in the guarded backend path, persistence in services/repositories, calculations in backend calculators, and labels/formatting in the frontend. Register a new Nest module in `backend/src/app.module.ts` only when it is an active feature.

31. Legacy code

Legacy means code no longer part of the active application but preserved locally for historical/reference purposes. It is not deleted, bundled, registered, or executed by normal application paths.

Archived frontend code includes the deprecated `app-data-api.js` shim and the disconnected alternate AI renderer/debug/stage branch under `frontend/legacy/`. The active replacement is the config/response API layer and the active AI modal/stream renderer.

Archived backend code includes the one-time `ResponseMatrix` converter, manual Prisma/bootstrap scripts, and the disconnected `app-data` module under `backend/legacy/`. Current schema work uses `backend/prisma/schema.prisma` and the package Prisma commands.

Never import legacy code into active source. Never run an archived migration against a live database casually. Ignored does not mean deleted, and `.gitignore` does not untrack files already tracked by Git. Before permanent deletion, repeat the import, script, database, deployment, and history audit.

32. Troubleshooting

“The results are empty”

Check whether the assessment has responses, whether responses belong to the tenant/assessment, and whether their status is `EVALUATED`. A `GRADED` answer-key row is not enough for the results service. Then check the answer key and assessment scope.

“The AI is not responding”

Check provider environment settings (`AI_PROVIDER` , `AI_PROVIDER_URL` , `AI_MODEL` , optional key), model availability, backend logs, and the fallback response. An unconfigured provider intentionally uses rule routing; a provider outage should not change grades.

“An OMR result looks wrong”

Inspect the image/scan result, detected choice and confidence/state, the verification queue, the answer key, and final response status. Teacher review is the safety boundary for ambiguous/multiple-mark results.

“Configuration disappeared”

Check `GET /api/config` , tenant/session context, and backend database connectivity. The frontend clears stale cached configuration when the authoritative load fails; its empty safe shape prevents a crash but is not real school data.

“Prisma fails”

Run `npm run prisma:generate` and `npx prisma validate --schema prisma/schema.prisma` from `backend/` . Review the current schema. Do not use the archived manual scripts as ordinary setup.

33. Development commands

Run frontend commands in `frontend/` :

```
npm install
npm start
npm run build
npm test
```

Run backend commands in `backend/` :

```
npm install
npm run start:dev
npm run build
npm test -- --runInBand
npm run prisma:generate
npx prisma validate --schema prisma/schema.prisma
npm run prisma:push
npm run prisma:migrate:dev
npm run seed
```

Use `.env.example` files as the starting point. Never commit `.env`, secrets, provider keys, or generated build output. There is no separate lint script in either package at the time of this documentation.

34. Do not touch this casually

- **`.env` and provider/database secrets:** they can expose credentials or data.
- **Production database:** schema/data changes can be irreversible; use a reviewed migration/backup process.
- **Prisma relationships:** changing a relation can orphan students, responses, or tenant data.
- **Tenant filtering and session/CSRF/auth code:** a small omission can expose one school's records to another.
- **Deterministic calculators:** changing a threshold or rounding rule changes official results; add tests and update this handbook.
- **Assessment status/evaluation rules:** results intentionally require `EVALUATED` rows.
- **Migration history:** do not rewrite history or casually rerun archived scripts.
- **Legacy archives:** preserve their reference value and do not re-import them into production paths.
- **AI permission boundaries:** new tools must enforce tenant/role access and must not become a hidden grading authority.

35. Glossary

- **Assessment:** A named test for a subject, section, term, and school year.
- **MPS / passingMps:** Minimum percentage required to pass an assessment.
- **TOS:** Table of Specifications; a worksheet connecting intended competencies to assessment items.
- **Competency:** A learning area or skill used to group mapped questions.
- **Item:** One assessment question.
- **Distractor:** A wrong multiple-choice option.
- **Difficulty index (`p`):** The proportion of students answering an item correctly; higher means easier for that group.
- **Discrimination index:** Difference between upper- and lower-performing groups' item success rates.
- **Point-biserial:** Relationship between getting an item right and performance on the remaining items.
- **Mastery:** Percentage of mapped competency responses answered correctly.
- **Percentile:** Relative position inside this evaluated assessment group, not a national norm.
- **Rank:** Dense score order; tied scores share a rank.
- **OMR:** Optical Mark Recognition; reading marked bubbles from an answer form.
- **Response:** A student's assessment-level answer/score record.
- **ResponseItem:** One student's answer and correctness row for one item.
- **Tenant:** School/organization boundary used for data isolation.
- **API:** A defined request/response interface between frontend and backend.
- **Frontend:** Browser screens and interaction code.

- **Backend:** Server code responsible for security, persistence, calculations, and orchestration.
- **Prisma:** The backend library that maps TypeScript database operations to the schema/database.
- **AI:** Optional language-model assistant and tool orchestrator; not the grade authority.
- **SSE:** Server-Sent Events, the streaming response format used by `/api/ai/stream`.
- **Remediation:** Follow-up practice/support proposed for a learning weakness.
- **Reassessment:** A later assessment proposed or created to check growth.

36. Source of truth

```
Database rows
    ↓
Backend services and deterministic calculators
    ↓
Authenticated API
    ↓
Frontend stores/views
    ↓
User interface
```

The database is authoritative for persisted tenant, assessment, response, and workflow records. Backend services/calculators are authoritative for deterministic values derived from those records. The frontend is the interaction and presentation layer. **AI is not the source of truth for grades, correctness, pass/fail, rankings, or psychometric calculations.**

37. Documentation rules

1. Explain the product before explaining the code.
2. Explain mathematics in human language before formulas.
3. Give every formula a real example and edge cases.
4. Give every major feature its actual implementation path.
5. Never invent functionality, formulas, routes, or AI capabilities.
6. Never describe archived code as active.
7. If code and documentation disagree, inspect code and document actual behavior.
8. Document questionable behavior or implementation bugs honestly.
9. Update this handbook when behavior, calculations, AI boundaries, or source-of-truth rules change.

The former `PROJECTKIT_DOCUMENTATION.md` and `CALCULATIONS.md` remain available as specialized/transition references, but this file is the single primary human-readable handbook. `PROJECT_KIT_FRONTEND_LOGIC.md` and backend-specific notes should link here when they describe overlapping behavior.